

UHabitacional

Diagrama de Arquitectura

Solucion en capas: ASP.NET Core MVC + Web API REST + SQL Server

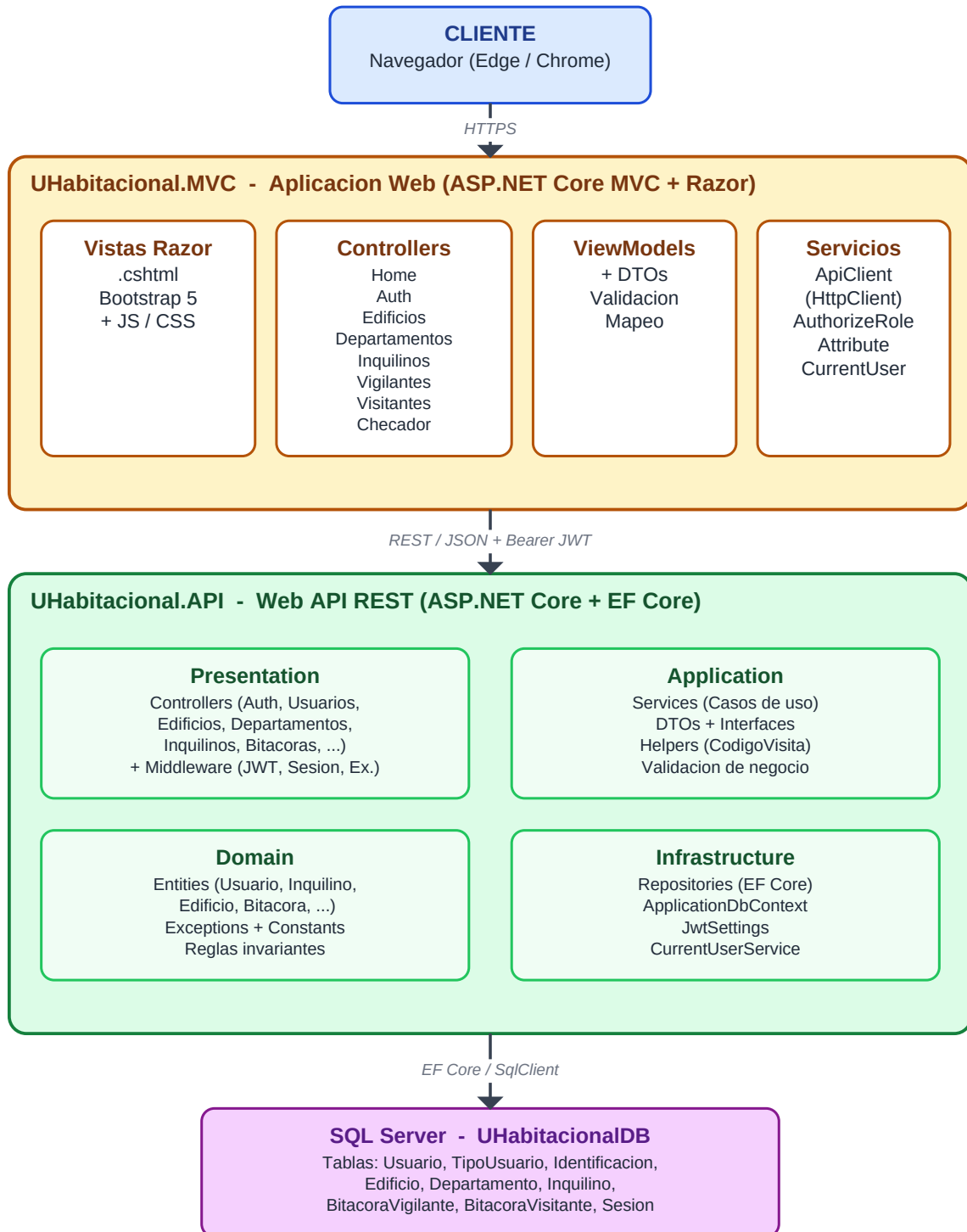


Diagrama de arquitectura - UHabitacional

Solucion en capas: cliente web (MVC) consume API REST (Clean Architecture) sobre SQL Server

Figura 1. Vista logica de la solucion. Cada bloque representa un proyecto o capa, y las flechas indican el sentido de la dependencia y el protocolo de comunicacion.

1. Vision general

La solución **UHabitacional** está organizada como una solución de Visual Studio (`UHabitacional.slnx`) que contiene dos proyectos principales: **UHabitacional.API** (Web API REST) y **UHabitacional.MVC** (aplicación web Razor), respaldados por una base de datos **SQL Server** (`UHabitacionalDB`). La separación responde al patrón cliente-servidor: el MVC es un cliente HTTP del API, y el API es la única puerta de acceso a la base de datos.

El API sigue el patrón **Clean Architecture** (Arquitectura limpia / Onion Architecture), dividido en cuatro capas concéntricas: *Domain* -> *Application* -> *Infrastructure* -> *Presentation*. La dirección de las dependencias siempre apunta hacia el dominio, garantizando que las reglas de negocio no dependan de detalles técnicos (EF Core, JWT, ASP.NET).

2. Descripción de cada capa

2.1 Cliente (Navegador)

El usuario final (Administrador, Vigilante o Inquilino) interactúa con la aplicación exclusivamente a través de un navegador moderno. Toda la comunicación entre el navegador y el servidor MVC se realiza sobre **HTTPS** y la sesión del usuario se mantiene mediante una **cookie de autenticación** emitida por el MVC tras un login exitoso.

2.2 UHabitacional.MVC (capa de presentación del cliente)

Es la aplicación web servida al usuario. Sus piezas principales son:

- **Vistas Razor (.cshtml)**: renderizan las pantallas (Inquilinos, Edificios, Departamentos, Vigilantes, Visitantes, Checador, Login, etc.) con Bootstrap 5 y JS mínimo.
- **Controllers**: orquestan cada caso de uso del UI; no contienen lógica de negocio, delegan al `ApiClient` y mapean DTOs a `ViewModels`.
- **ViewModels y Models / ApiDtos**: representaciones específicas para las vistas (con sus validaciones de `DataAnnotations`) y los DTOs serializados que viajan al API.
- **ApiClient**: servicio tipado sobre `HttpClient` que inyecta el token JWT del usuario actual en el header `Authorization: Bearer` antes de cada llamada al API.
- **AuthorizeRoleAttribute / CurrentUser**: filtros propios que aplican autorización por rol (Administrador, Vigilante, Inquilino) en cada acción del MVC.

2.3 UHabitacional.API (Web API REST)

Es el corazón del sistema. Expone endpoints REST en formato JSON, valida tokens JWT y ejecuta toda la lógica de negocio. Internamente está organizado en cuatro capas:

a) Presentation

- **Controllers** (`AuthController`, `UsuariosController`, `EdificiosController`, `DepartamentosController`, `InquilinosController`, `BitacoraVigilanteController`, `BitacoraVisitanteController`, `SesionController`, etc.): traducen HTTP a DTOs y delegan a la capa de aplicación.
- **Middleware**: pipeline ASP.NET Core que incluye autenticación JWT, validación de sesión (consulta la tabla `Sesion` en cada request para verificar que el JTI siga activo) y manejo global de excepciones traducidas a respuestas HTTP coherentes (400, 401, 403, 404, 409, 500).

b) Application

- **Services:** implementan los casos de uso (AuthService, InquilinoService, BitacoraVisitanteService, ...). Aqui viven las validaciones de negocio (un departamento no puede exceder los pisos del edificio, un inquilino activo debe tener FechaFin = null, un vigilante no puede registrar dos turnos abiertos a la vez, etc.).
- **DTOs:** contratos de datos que el API expone (entrada y salida).
- **Interfaces** (IRepository, IService): abstracciones que permiten invertir la dependencia de la capa de aplicacion sobre la infraestructura.
- **Helpers:** utilidades sin estado, por ejemplo CodigoVisitaGenerator, que produce el codigo aleatorio de 6 caracteres (mayuscula + minuscula + numero) usado en visitas.

c) Domain

- **Entities:** clases POCO que modelan el dominio (Usuario, TipoUsuario, Identificacion, Edificio, Departamento, Inquilino, BitacoraVigilante, BitacoraVisitante, Sesion) con DataAnnotations para mapeo a SQL.
- **Exceptions:** BusinessException, NotFoundException, ValidationException, ForbiddenOperationException: tipos de error de negocio que el middleware traduce a codigos HTTP.
- **Constants:** RolesUsuario (Administrador, Vigilante, Inquilino) para evitar strings magicos.

d) Infrastructure

- **ApplicationDbContext:** contexto de Entity Framework Core que mapea las entidades a las tablas de SQL Server, configura indices unicos (Email, JTI), claves foraneas y la cascada.
- **Repositories:** implementaciones concretas (UsuarioRepository, InquilinoRepository, ...) que encapsulan las consultas LINQ contra el DbContext.
- **JwtSettings:** configuracion fuerte-tipada (issuer, audience, secret, expiracion) leida desde appsettings.json.
- **CurrentUserService:** implementa ICurrentUserService y expone el usuario autenticado (Id, rol, claims) extraidos del HttpContext.

2.4 SQL Server (UHabitacionalDB)

Persistencia relacional. Los scripts se ejecutan en orden desde la carpeta /Database: 01_CreateDatabase, 02_CreateTables, 03_SeedData, 05_AddSesionTable. Incluye nueve tablas, claves foraneas y un indice unico sobre Usuario.Email y Sesion.Jti.

3. Patrones y principios aplicados

Patron / Principio	Como se aplica en el proyecto
Clean Architecture	El API se divide en Domain / Application / Infrastructure / Presentation; las dependencias apuntan hacia el dominio.
Repository + UoW	Cada entidad relevante tiene su IRepository y EF Core actua como Unit of Work via SaveChanges.
Dependency Injection	Todas las interfaces (servicios, repositorios, CurrentUserService) se registran en Program.cs y se inyectan por constructor.

DTO + Mapping	Los Controllers nunca exponen entidades; cada request/response usa DTOs específicos.
Middleware Pipeline	Autenticacion JWT, validacion de sesion y manejo global de excepciones se aplican como middlewares en cadena.
Soft Delete	Las entidades sensibles (Usuario, Inquilino, Identificacion, BitacoraVisitante) usan el flag Activo para borrado logico.
Auditoria temporal	Todas las tablas relevantes mantienen FechaCreacion y FechaModificacion.
JWT + Sesion en BD	El JWT contiene el claim 'jti' que se almacena en la tabla Sesion; el middleware lo valida en cada request, permitiendo revocar tokens.

4. Flujo de una peticion tipica

A continuacion se describe paso a paso lo que ocurre cuando un Administrador da de alta un nuevo Inquilino desde la aplicacion MVC:

1. El administrador llena el formulario en la vista `Razor Views/Inquilinos/Form.cshtml` y envia el submit.
2. `InquilinosController.Create(...)` del MVC recibe el POST, valida el ViewModel y construye un `InquilinoCreateDto`.
3. El controller llama a `ApiClient.PostAsync`, que adjunta el JWT del usuario y envia la peticion HTTPS al API.
4. El **middleware JWT** del API valida la firma y la expiracion del token; el **middleware de Sesion** consulta la tabla Sesion para comprobar que el JTI siga activo.
5. `InquilinosController.Create` del API recibe el DTO y lo entrega a `InquilinoService.CreateAsync`.
6. El service ejecuta validaciones de negocio (el usuario es de tipo Inquilino, el departamento existe y no esta ocupado por otro inquilino activo, etc.).
7. Si todo es valido, el service usa `IInquilinoRepository` para hacer `Add + SaveChangesAsync`; EF Core genera el INSERT y obtiene el Id.
8. El service mapea la entidad a `InquilinoDto` y el controller responde `201 Created`.
9. El `ApiClient` deserializa el DTO y el MVC redirige al usuario a `Inquilinos/Index`, donde ve la lista actualizada.

5. Pila tecnologica

Capa	Tecnologia
Cliente	.NET + Razor Views + Bootstrap 5 + JavaScript
MVC	ASP.NET Core MVC, Cookie Authentication, HttpClient
API	ASP.NET Core Web API, JWT Bearer, Swagger / OpenAPI
ORM	Entity Framework Core (Code-First con DataAnnotations)
BD	SQL Server 2019+ (scripts en /Database)

Seguridad	JWT firmado HS256, sesion persistida en BD (revocacion server-side)
Empaquetado	Solucion .slnx con UHabitacional.API y UHabitacional.MVC